

Experiments on KVM-based system call virtualization

Tesi di Laurea in Informatica

Relatore: Renzo Davoli, **Laureando:** Fabio Murer

Alma Mater Studiorum - Università di Bologna

31/10/2025

Obiettivo

Le VM di sistema sfruttano l'hardware per essere efficienti, ma sono monolitiche e dispendiose. Le VM di processo sono leggere e modulari, ma dipendono da meccanismi software che ne limitano le prestazioni.

- **Utilizzare la virtualizzazione hardware** messa a disposizione dal modulo kernel Linux KVM **come tecnica di virtualizzazione di processo**
- Creare un **prototipo** in grado di eseguire semplici programmi Linux in un ambiente virtuale
- **Confrontare le prestazioni** rispetto alle metodologie (software) esistenti

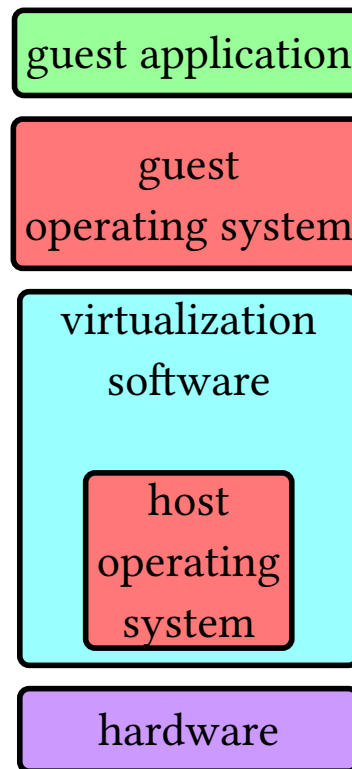
Machine virtuali

Implementano strati di astrazione che permettono di astrarre o ridefinire le interfacce (hardware e software) sottostanti

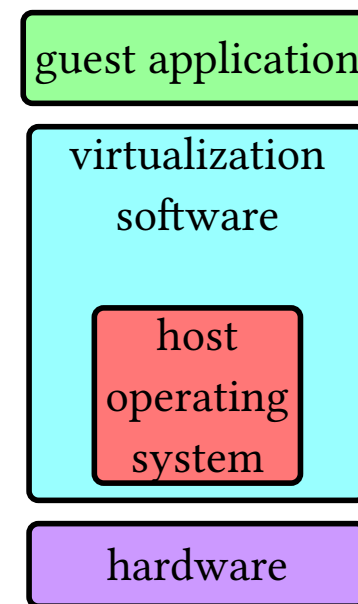
Hypervisor: il gestore di macchine virtuali

Tipologie

- A livello di Sistema
- A livello di Processo



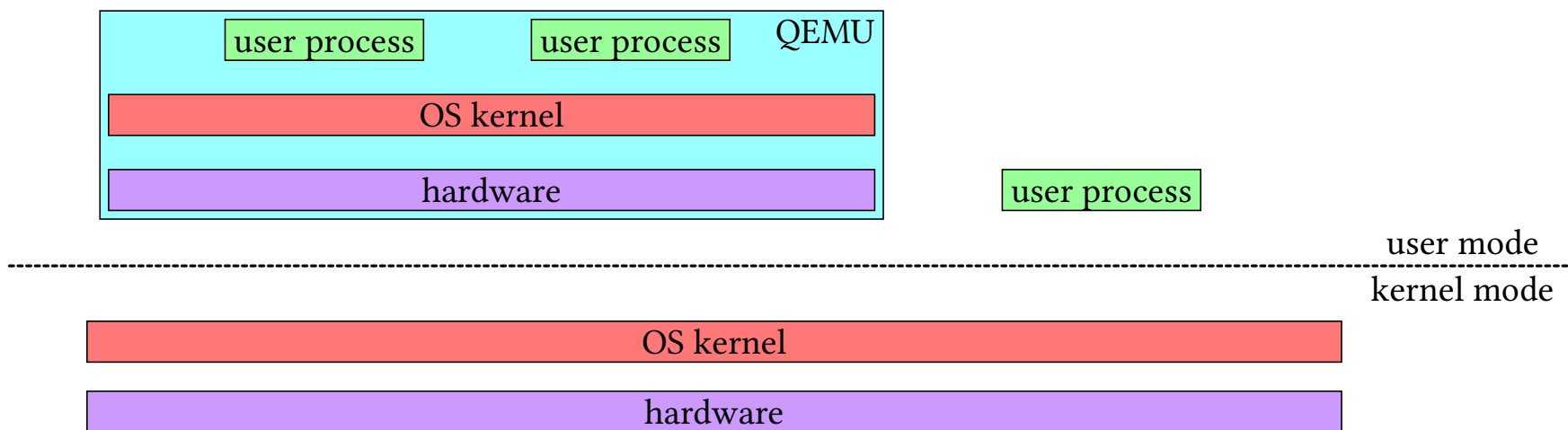
System VM



Process VM

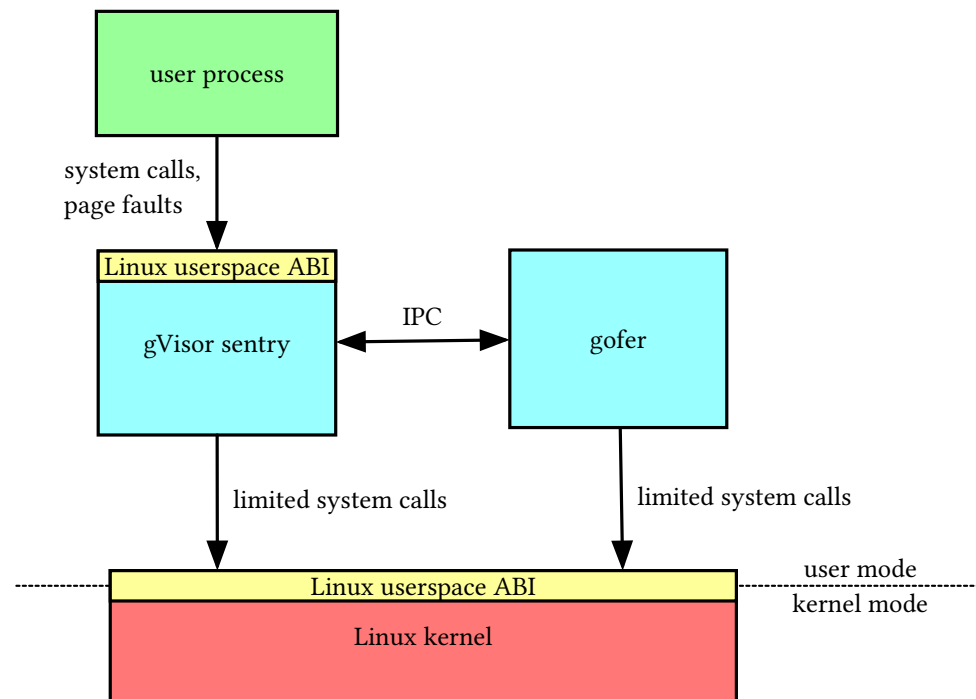
Macchine virtuali a livello di Sistema

- Possono eseguire sistemi operativi differenti dal sistema host
- Consumo di risorse più elevato
- Grado di isolamento più alto



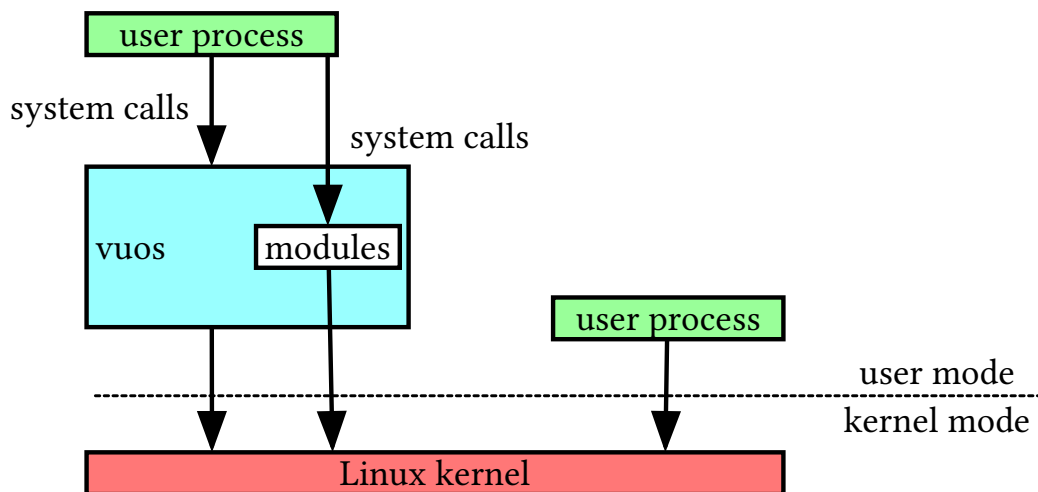
Macchine virtuali a livello di Processo

- Utilizzano meno risorse
- Capacità di virtualizzazione modulare
 - Virtualizzazione parziale



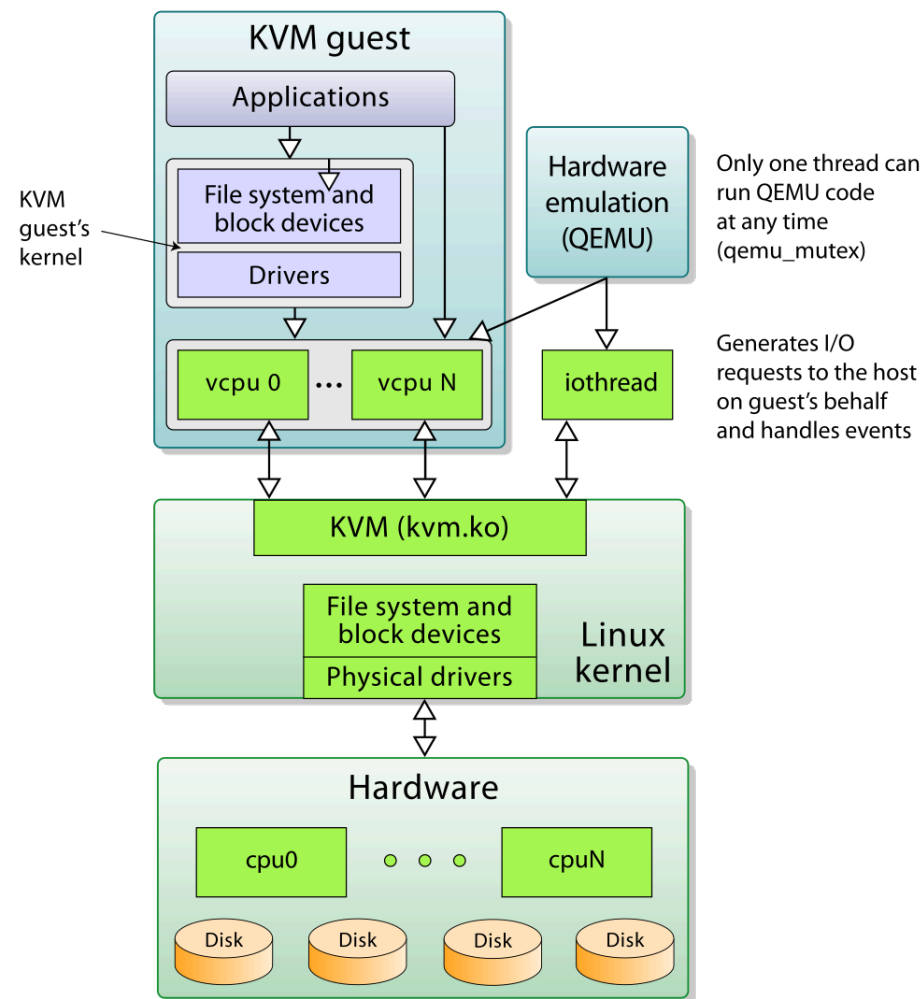
Tecniche di virtualizzazione di processo

- Basate sulla **cattura di system call**
- Dopo la cattura, ne viene modificata la semantica, eseguita direttamente dall'hypervisor o delegata al kernel sottostante
- Tecniche software come: **ptrace**, **seccomp-bpf**
- L'hypervisor può **alterare la percezione della realtà** del processo virtualizzato **cambiando la semantica delle system call**



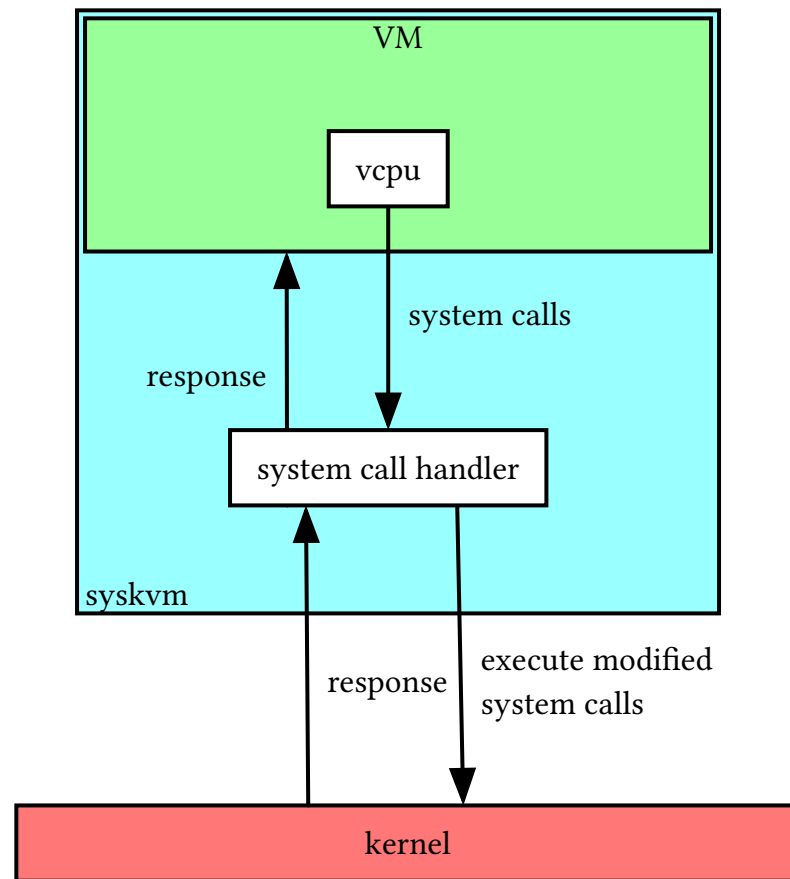
KVM

- Modulo kernel Linux
- Utilizza estensioni **hardware del processore** (Intel VT-x, amd AMD-V) che permettono all'hypervisor di **controllare direttamente la CPU**, invece di emularla via software
- Prestazioni superiori
- Utilizzato nella virtualizzazione di sistema



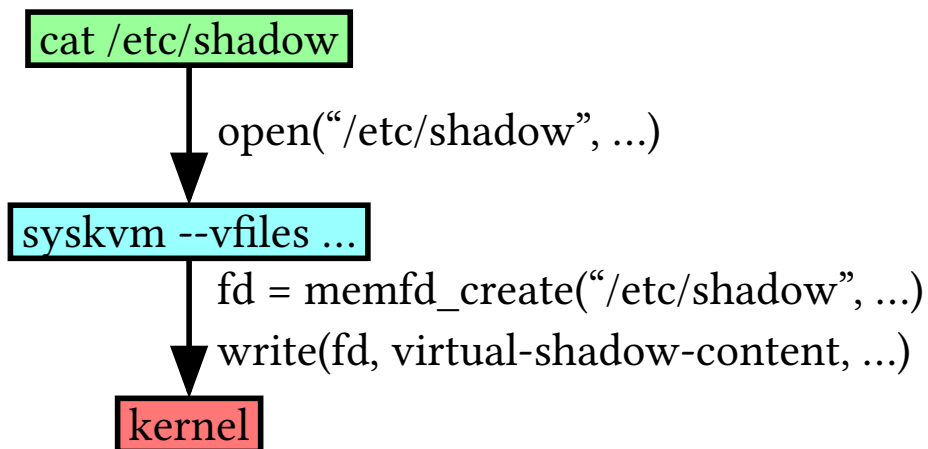
Il prototipo: Syskvm

- Esegue un processo in una **macchina virtuale KVM senza kernel guest**
- L'esecuzione di una **system call** comporta l'uscita dalla **macchina virtuale** restituendo il controllo all'hypervisor che soddisfa la richiesta implementandola o delegandola al kernel sottostante
- Meccanismo di cattura hardware



Risultato

- Esecuzione di normali programmi Linux in una macchina virtuale KVM
- Virtualizzare risorse attraverso la modifica di system call (files)



```
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1291 Jul 25 11:12 /
etc/shadow
```

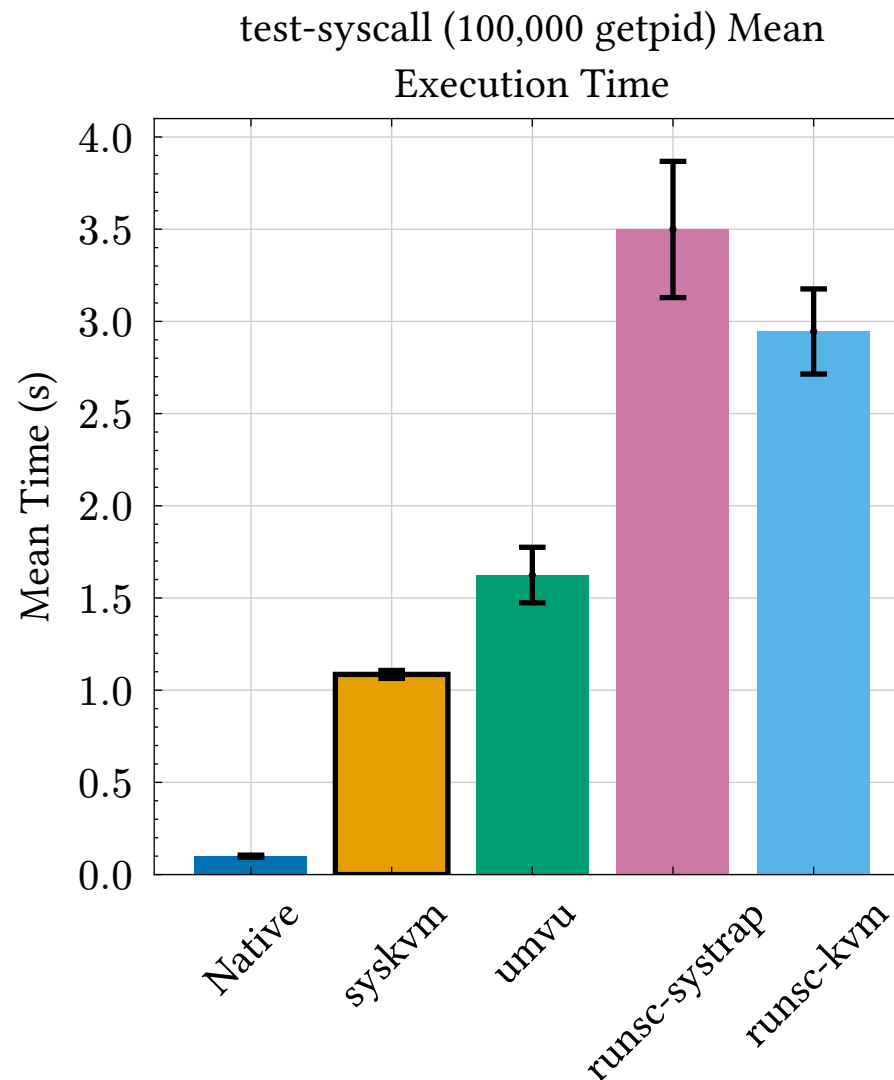
```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

```
$ ./syskvm --vfiles -- cat /etc/shadow
root:
$6$abcdefgh$1234567890abcdefghijlmnopqrstuvwx
:19000:0:99999:7:::
daemon*:19000:0:99999:7:::
bin*:19000:0:99999:7:::
sys*:19000:0:99999:7:::
sync*:19000:0:99999:7:::
games*:19000:0:99999:7:::
man*:19000:0:99999:7:::
lp*:19000:0:99999:7:::
mail*:19000:0:99999:7:::
syskvm:
$6$ijklmnop$0987654321ponmlkjihgfedcbazyxwvutsr
:19000:0:99999:7:::
```

Prestazioni

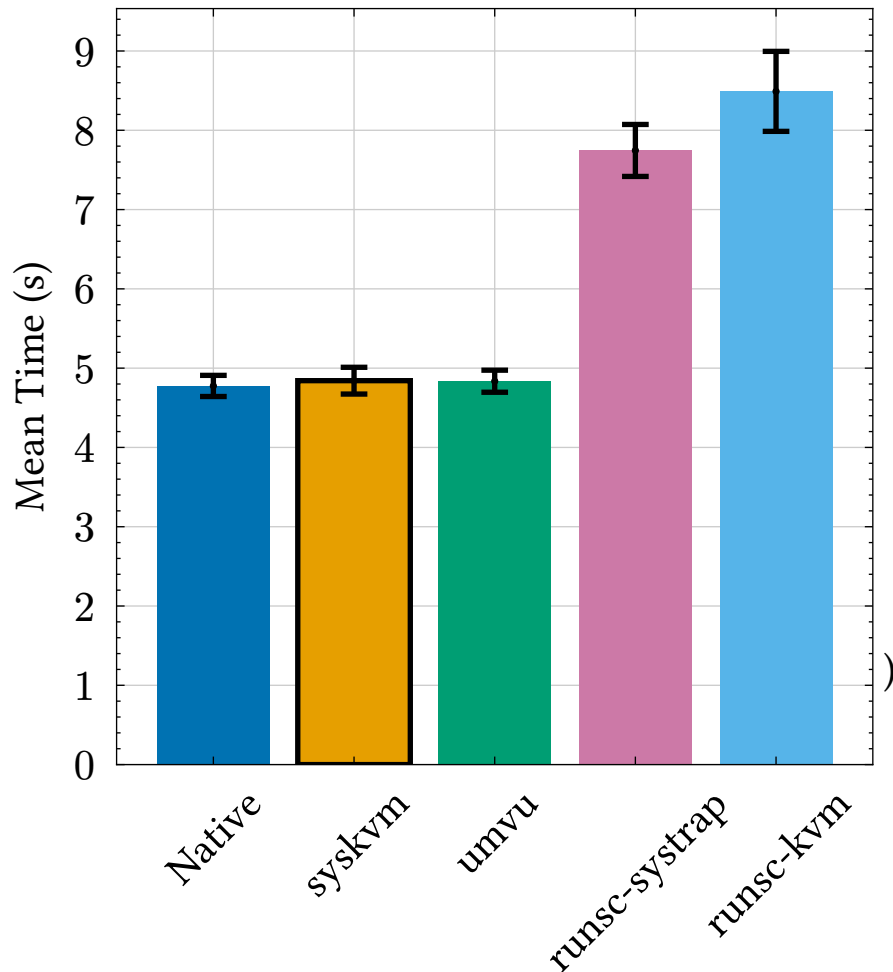
Confronto delle prestazioni tra

- Esecuzione **nativa**
- **VUOS** (umvu): Seccomp-bpf
- **gVisor**: (runsc-systrap):
seccomp-bpf, (runsc-kvm):
KVM, sentry come kernel

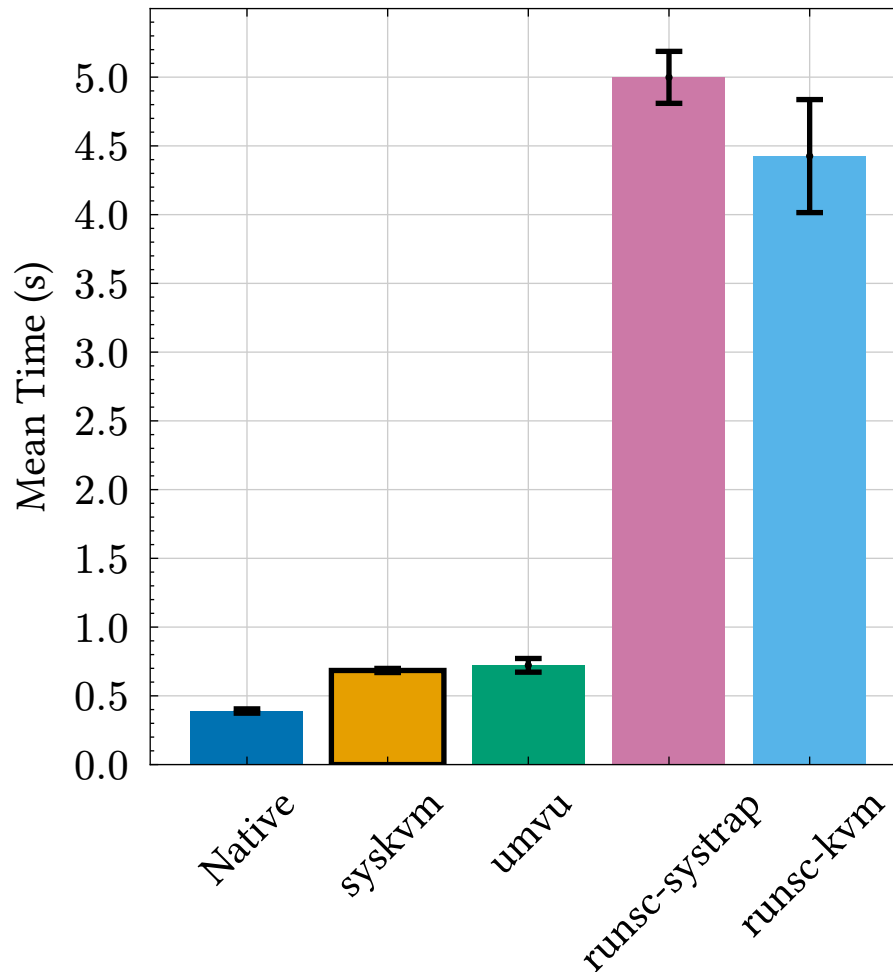


Prestazioni

test-cpu (2,500,000,000 subtractions) Mean
Execution Time



test-io (10,000 * 8KB file reads and writes)
Mean Execution Time



Conclusioni

- Miglioramento delle **prestazioni**
 - **1.5 volte** più veloce rispetto a umvu nella cattura di system call
- Maggiore **sicurezza** grazie all'utilizzo di meccanismi hardware
 - Riduzione della superficie di attacco, meno codice kernel utilizzato
- Maggiore **espressività**: capacità di eseguire istruzioni privilegiate

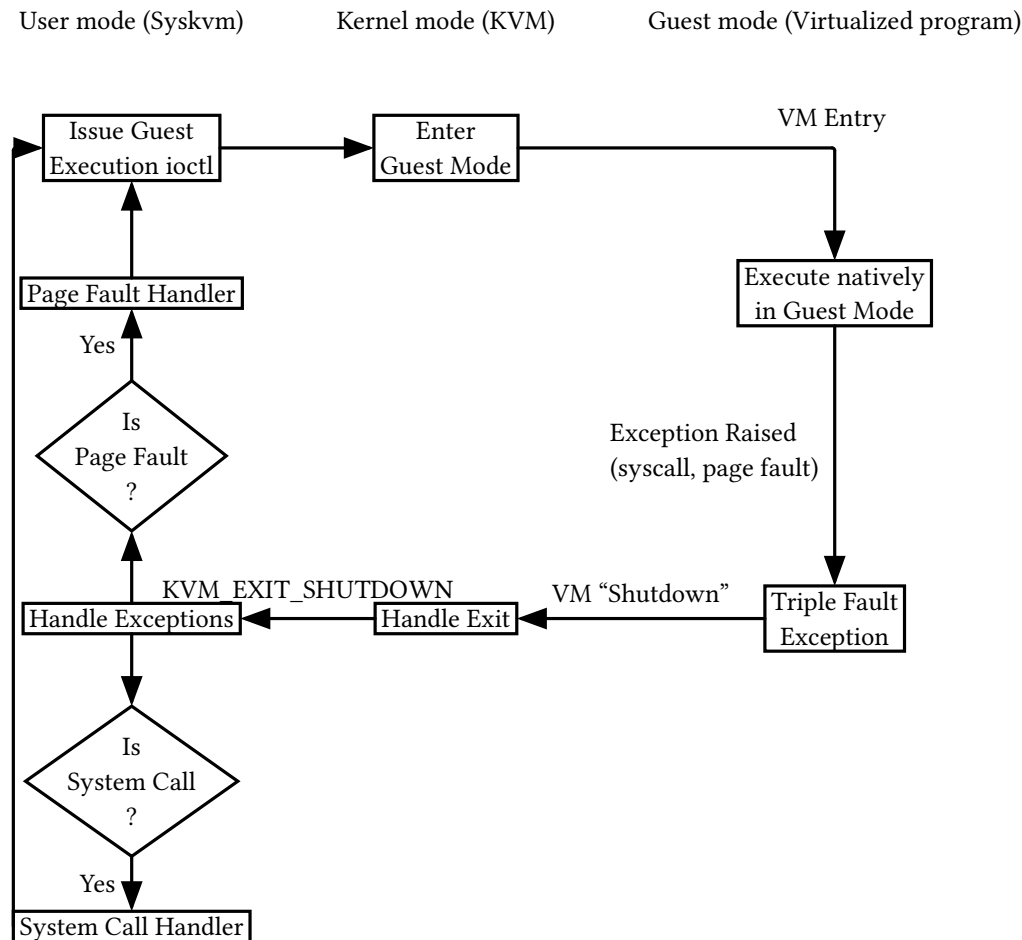
Sviluppi futuri

- Estendere numero di system call supportate:
 - Multithreading: clone, exec, ...
 - Gestione dei segnali: kill, raise, pause, ...
- Integrazione con VUOS
- Creazione di un runtime di immagini OCI: esecuzione di containers

Grazie per l'attenzione

Implementazione

- Inizializzare la macchina virtuale
 - CPU: sse, avx
 - Memoria: Paginazione
- Gestire **eccezioni**
 - Page faults: meccanismo di demand paging
- Implementare sottoinsieme minimale di **system call** (brk, mmap, open, read, write, ...)
- Server GDB per permettere il debugging dei programmi virtualizzati



test-syscall.c

```
#include <asm/unistd_64.h>
#include "syscall.h"

#define ITERATIONS 100000

void __attribute__((noreturn)) __attribute__((section(".start"))) _start(void) {

    for (int i = 0; i < ITERATIONS; i++) {
        syscall_syscall(__NR_getpid, 0, 0, 0, 0, 0, 0);
    }

    syscall_syscall(__NR_exit_group, 0, 0, 0, 0, 0, 0); // exit success
    for (;;) {
        __asm__("hlt");
    }
}
```


test-cpu.c

```
#include "syscall.h"
#include <asm/unistd_64.h>

#define ITERATIONS 50000

void __attribute__((noreturn)) __attribute__((section(".start"))) _start(void) {
    long x = 0;
    for (long i = 0; i < ITERATIONS; i++) {
        for (long j = 0; j < ITERATIONS; j++) {
            x = i - j;
        }
    }

    syscall_syscall(__NR_exit_group, x, 0, 0, 0, 0, 0);
    for (;;) {
        __asm__("hlt");
    }
}
```

test-io.c

```
#define ITERATIONS 10000
#define BUFFER_SIZE (8 * 1024)

//...
// Read from urandom and write to /tmp/trash 10000 times
for (int i = 0; i < ITERATIONS; i++) {
    // Read 8KB from urandom
    long bytes_read =
        syscall_syscall(__NR_read, urandom_fd, (long)buffer, BUFFER_SIZE, 0, 0, 0);

    // Write to /tmp/trash
    long bytes_written =
        syscall_syscall(__NR_write, trash_fd, (long)buffer, bytes_read, 0, 0, 0);
}
//...
```